

資料結構

Data Structure

Lab11

姓名：李品融

學號：113310217

Lab11-Q1

這題要求我們實作二元樹的 BFS 層序走訪，依輸入層數計算該層所有節點值總和，並加入防禦性編程機制處理超出樹高或非法輸入之異常。

Code

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

const int EMPTY = NULL; // 用 NULL 代表沒有節點

class TreeNode {
public:
    int value; // 節點值
    TreeNode* left; // 左子節點
    TreeNode* right; // 右子節點

    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {} // 初始化節點
};

class BinaryTree {
public:
    TreeNode* root; // 樹根節點

    BinaryTree() : root(nullptr) {}

    TreeNode* buildTree(const vector<int>& arr) { // 建立樹
        if (arr.empty() || arr[0] == EMPTY) return nullptr;

        queue<TreeNode*> q; // 建立 queue 儲存待處理的節點指標
        root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
        q.push(&root); // 將根節點的指標加入 queue

        size_t i = 1; // 陣列索引
        while (!q.empty() && i < arr.size()) {
```

```

TreeNode** nodePtr = q.front();// 取出 queue 的第一個節點指標
q.pop();// 將該節點從 queue 中刪除

// 左子節點
if (i < arr.size()) {
    if (arr[i] != EMPTY) {
        (*nodePtr)->left = new TreeNode(arr[i]);// 添加左子節點
        q.push(&((*nodePtr)->left));// 將左子節點的指標加入 queue
    }
    i++;
}

// 右子節點
if (i < arr.size()) {
    if (arr[i] != EMPTY) {
        (*nodePtr)->right = new TreeNode(arr[i]);// 添加右子節點
        q.push(&((*nodePtr)->right));// 將右子節點的指標加入 queue
    }
    i++;
}
}

return root;
}

void Depth_first_search(TreeNode* node) {//深度優先搜尋
if (node == nullptr) return;
cout << node->value << " ";// 訪問當前節點的值
Depth_first_search(node->left);// 遞迴遍歷左子樹
Depth_first_search(node->right);// 遞迴遍歷右子樹
}

void Breadth_first_search(TreeNode* root) {
if (root == nullptr) return;
queue<TreeNode*> q;//建立 queue 儲存待處理的節點指標
q.push(root); // 將根節點的指標加入 queue

while (!q.empty()) {

```

```

    TreeNode* current = q.front();// 取出 queue 的第一個節點指標
    q.pop(); // 將該節點從 queue 中刪除
    cout << current->value << " ";
    if (current->left) q.push(current->left); // 將左子節點的指標加入 queue
    if (current->right) q.push(current->right); // 將右子節點的指標加入 queue
}
}

// 計算特定層數的節點數值總和
void printLevelSum(int target_level) {
    // 數的層數（索引）是從 0 開始的正整數
    // 若使用者輸入小於零的數，將先被攔截
    if (target_level < 0) {
        cout << "Invalid layer number. Layer must be >= 0." << endl;
        return;
    }

    // 檢查空樹狀況
    // 必須先檢查這棵樹在呼叫此函數時是否還沒被建立（意旨 root 指向 nullptr）
    // 若不檢查，後續的程式碼會把 nullptr 放進佇列中
    // 當程式試圖從佇列取出指標並存取 current->value 時，就會導致程式碼當機，無法運行
    if (root == nullptr) {
        cout << "The tree is empty." << endl;
        return;
    }

    // 建立一個標準的 FIFO 佇列 Queue，儲存接下來要造訪的節點指標
    queue<TreeNode*> q;
    q.push(root); // 將根節點推入佇列，作為走訪的起點
    int current_level = 0; // 初始化當前層數計數器。因為從 0 開始算起，所以根節點
    // root 所在的層數就是第 0 層

    // 只要佇列裡面還有節點，就代表樹還沒有被完全走訪完畢，繼續執行迴圈
    // q.size() 代表的是當前這一層的節點總數
    while (!q.empty()) {
        int level_size = q.size(); // 用一個變數將 q.size() 儲存起來
        int level_sum = 0; // 初始化當前層的累加總和變數為 0

```

```

// 內層迴圈，只處理當前層的節點
// 迴圈只會執行 level_size 次，確保把當前層的節點全部吐出來，而不會混到下一層
for (int i = 0; i < level_size; ++i) {
    TreeNode* current = q.front(); // 取出佇列最前端的節點

    q.pop(); // 將該節點移出佇列

    level_sum += current->value; // 將目前取出的節點數值，累加到當前層的總和中

    if (current->left) q.push(current->left); // 若左子節點存在，推入佇列
    if (current->right) q.push(current->right); // 若右子節點存在，推入佇列
}

// 找到目標層數，輸出結果並結束函數
// 當內層迴圈結束，代表某一整層已經完全計算完畢
// 檢查目前的層數 (current_level) 是否為輸入的目標 (target_level)
if (current_level == target_level) { // 格式完全符合題目要求
    cout << "The sum of level " << target_level << " is: " << level_sum <<
endl;
    return;
}

current_level++; // 如果目前層數不是目標，則將層數計數器加 1，準備處理下一層
}

// queue 空了卻還沒對應到 target_level，代表超出樹高
// 當 while 迴圈因為 q.empty() 而自然結束時，代表整棵二元樹所有的節點都已經被
// 徹底走訪完畢了。如果程式執行到了這行，卻還沒有在上面的 if (current_level ==
target_level) 觸發 return
// 代表使用者輸入的 target_level 超出了這棵樹實際上的最大高度
cout << "The layer exceeds the tree height." << endl;
}
};

int main() {
    BinaryTree tree;

```

```

vector<int> arr = { 1, 2, 3, 4, 5, 6, 7, 8, 9, NULL, NULL, 10, 11, NULL,
NULL };
tree.buildTree(arr);

cout << "DFS Result: ";
tree.Depth_first_search(tree.root);
cout << endl;

cout << "BFS Result: ";
tree.Breadth_first_search(tree.root);
cout << endl;

// 互動式輸入區塊
int input_layer;
cout << "Please enter the layer to query, starting from 0: ";

// 確保使用者輸入的是合法的整數，避免輸入英文等非數字導致無窮迴圈或崩潰
if (cin >> input_layer) {
    tree.printLevelSum(input_layer); // 輸入合法，執行計算
} else {
    cout << "Invalid input! Please enter an integer." << endl; // 輸入非法（如英文
字元），進行報錯提示
}

system("pause");
return 0;
}

```

Result

若輸入 2，則輸出: The sun of level 2 is: 22

```
C:\Users\user\資料庫\學校資料 > x + v
DFS Result: 1 2 4 8 9 5 3 6 10 11 7
BFS Result: 1 2 3 4 5 6 7 8 9 10 11
Please enter the layer to query, starting from 0: 2
The sum of level 2 is: 22
請按任意鍵繼續 . . . |
```

```
Lab11.cpp
1 #include <iostream>
2 #include <queue>
3 #include <vector>
4 using namespace std;
5
6 const int EMPTY = NULL; // 用 NULL 代表沒有節點
7
8 class TreeNode {
9 public:
10 int value; // 節點值
11 TreeNode* left; // 左子節點
12 TreeNode* right; // 右子節點
13
14 TreeNode(int val) : value(val), left(nullptr), right(nullptr) {}
15 };
16
17 class BinaryTree {
18 public:
19 TreeNode* root; // 樹根節點
20
21 BinaryTree() : root(nullptr) {}
22
23 TreeNode* buildTree(const vector<int>& arr) {
24 if (arr.empty() || arr[0] == EMPTY) return nullptr;
25
26 queue<TreeNode*> q; // 建立queue儲存待處理的節點指標
27 root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
28 q.push(&root); // 將根節點的指標加入queue
29
30 while (!q.empty()) {
31     TreeNode* node = q.front();
32     if (node == nullptr) continue;
33     node->left = new TreeNode(arr[q.front()-1]);
34     node->right = new TreeNode(arr[q.front()+1]);
35     q.push(node->left);
36     q.push(node->right);
37     q.pop();
38 }
39 return root;
40 }
41
42 int main() {
43     vector<int> arr = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11};
44     BinaryTree tree;
45     tree.root = tree.buildTree(arr);
46
47     // DFS
48     vector<int> dfsResult;
49     dfsResult = tree.dfs();
50     cout << "DFS Result: ";
51     for (int i = 0; i < dfsResult.size(); i++) {
52         cout << dfsResult[i] << " ";
53     }
54     cout << endl;
55
56     // BFS
57     vector<int> bfsResult;
58     bfsResult = tree.bfs();
59     cout << "BFS Result: ";
60     for (int i = 0; i < bfsResult.size(); i++) {
61         cout << bfsResult[i] << " ";
62     }
63     cout << endl;
64
65     int layer;
66     cout << "Please enter the layer to query, starting from 0: ";
67     cin >> layer;
68
69     int sum = 0;
70     if (layer < 0 || layer > 10) {
71         cout << "Invalid layer input." << endl;
72     } else {
73         sum = tree.sumOfLevel(layer);
74         cout << "The sum of level " << layer << " is: " << sum << endl;
75     }
76
77     return 0;
78 }
```

若輸入 4，則輸出: The layer exceeds the tree height.

```
C:\Users\user\資料庫\學校資料  X  +  v
DFS Result: 1 2 4 8 9 5 3 6 10 11 7
BFS Result: 1 2 3 4 5 6 7 8 9 10 11
Please enter the layer to query, starting from 0: 4
The layer exceeds the tree height.
請按任意鍵繼續 . . .
```

The image shows a C++ IDE with a source code editor on the left and a console window on the right.

Source Code (LAB11.cpp):

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

const int EMPTY = NULL; // 用 NULL 代表沒有節點

class TreeNode {
public:
    int value; // 節點值
    TreeNode* left; // 左子節點
    TreeNode* right; // 右子節點
    TreeNode(int val) : value(val), left(nullptr), right(nullptr) {}
};

class BinaryTree {
public:
    TreeNode* root; // 樹根節點
    BinaryTree() : root(nullptr) {}

    TreeNode* buildTree(const vector<int>& arr) { // 建立
        if (arr.empty() || arr[0] == EMPTY) return nullptr;
        queue<TreeNode*> q; // 建立queue 儲存待處理的節點指標
        root = new TreeNode(arr[0]); // 建立根節點 (陣列第一個元素)
        q.push(&root); // 將根節點的指標加入queue
    }
};
```

Console Output:

```
DFS Result: 1 2 4 8 9 5 3 6 10 11 7
BFS Result: 1 2 3 4 5 6 7 8 9 10 11
Please enter the layer to query, starting from 0: 4
The layer exceeds the tree height.
請按任意鍵繼續 . . .
```